

AC⚡DCompiler

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE - GII2

Nombre	Apellido	Legajo	E-mail
Isabel	Conde	65.779	isconde@itba.edu.ar
Tomas	Varettoni	64.200	tvarettoni@itba.edu.ar
Bruno	Taccone	64.888	btaccone@itba.edu.ar

1. Introducción	2
2. Modelo Computacional	2
2.1 Dominio	2
2.2 Lenguaje	3
3. Implementación	3
3.1 CircuiTikZ	4
3.2 Frontend	5
3.3 Backend	7
3.4 Dificultades Encontradas	9
4. Futuras Extensiones	9
5. Conclusiones	10
6. Referencias	10
7. Bibliografía	10

1. Introducción

Este informe documenta el proceso del diseño y desarrollo de un nuevo lenguaje, orientado a la descripción y graficación de circuitos electrónicos. El objetivo de este lenguaje es abstraer la complejidad de LaTeX, permitiendo a los usuarios simplemente declarar aquellos elementos que estén en el circuito para luego obtener el gráfico.

La composición de gráficos técnicos mediante herramientas como CircuiTikZ requiere posicionamiento manual de coordenadas y múltiples elecciones de diseño. Para solucionar esto, nuestra propuesta busca ofrecer una interfaz intuitiva y legible, que permita al usuario definir la estructura de un circuito mediante declaraciones sencillas, delegando en el compilador la tarea de traducir estas instrucciones a código LaTeX válido (que utiliza CircuiTikZ).

A lo largo de este documento se describe el ciclo de vida completo del compilador, el cual fue construido sobre el repositorio <https://github.com/agustin-golmar/Flex-Bison-Compiler>, provisto por la cátedra. El desarrollo abarca desde la etapa de Frontend, donde se implementaron los analizadores léxico y sintáctico utilizando las herramientas Flex y Bison, hasta el Backend, encargado del análisis semántico mediante tabla de símbolos y la generación final del código. Asimismo, se exponen las decisiones de diseño tomadas para delimitar el alcance del lenguaje, adaptando el dominio a las restricciones técnicas encontradas durante la implementación.

2. Modelo Computacional

El objetivo de este lenguaje es la obtención de código LaTeX para la graficación de circuitos electrónicos, a partir de un lenguaje mucho más intuitivo que permite describir un circuito en forma sencilla y legible.

2.1 Dominio

El dominio consiste en ciertos circuitos RLC, tanto de corriente alterna como directa. Cabe destacar que, si bien al diseñar el lenguaje el dominio comenzó siendo mucho más amplio, el desarrollo del mismo (tanto del Frontend como del Backend) nos planteó diversas limitaciones que debimos aplicar a nuestro dominio.

Los circuitos incluidos son aquellos que tienen una malla principal de cuatro lados, donde en cada lado hay máximo un elemento, el cual puede ser un componente o un paralelo. Los componentes pueden ser: fuente de corriente directa, fuente de corriente alterna, resistor (de distintos tipos), inductor, capacitor, interruptor, voltímetro, o amperímetro. Para aquellos lados cuyo elemento sea un paralelo, pueden tener múltiples ramas, donde cada una puede tener como máximo un componente de los listados anteriormente (excluyendo el paralelo).

El lenguaje permite la graficación de múltiples circuitos como los descritos anteriormente, independientes entre sí y todos con un identificador propio. Al mismo tiempo, las componentes declaradas dentro de un mismo circuito tienen etiquetas que las identifican (todas

diferentes entre sí) y también pueden tener un valor asignado. También se verifica que cada circuito tenga solamente un tipo de fuente (es decir, solamente con fuentes de corriente directa o alterna).

2.2 Lenguaje

Para representar este dominio, se diseñó y desarrolló un lenguaje declarativo: el usuario no explica cómo dibujar los objetos, sino que declara su existencia y el compilador es quien se encarga de este procesamiento y posicionamiento.

El lenguaje consiste de dos tipos de elementos: estructuras y componentes. Las estructuras son declaradas con su tipo, su identificador y seguidas por llaves; dentro de estas llaves se declara el contenido de la misma. Los componentes son declarados con su tipo, su identificador y su lista de parámetros (en caso de tenerlos) entre paréntesis. Dos elementos dentro de un mismo bloque son separados por comas.

La estructura fundamental del lenguaje es el bloque **Circuit**, pudiendo tener múltiples dentro de un mismo programa. La declaración del mismo ya implica la inclusión de la malla central. El contenido de esta son los componentes o paralelos que pudiera tener (respetando el máximo de cuatro).

Los posibles componentes y sus posibles parámetros son:

- **ACSource** - puede recibir un valor (real o imaginario) y un modificador para dicho valor.
- **DCSource** - se puede indicar la polaridad con ‘-|+’ o ‘+|-’, siendo la primera la opción por defecto. Luego, también puede recibir un valor y su modificador.
- **Resistor** - se debe especificar su tipo: **regular**, **potentiometer**, **thermistor** o **photoresistor**. Luego, como en los otros casos, se puede ingresar su valor y un modificador para el mismo.
- **Inductor** - se puede indicar un valor y un modificador.
- **Capacitor** - se puede indicar la polaridad de la misma manera que en el **DCSource**, y también se puede agregar un valor y un modificador.
- **Voltmeter** - no recibe parámetros.
- **Ammeter** - no recibe parámetros.
- **Switch** - se debe indicar si está abierto (**open**) o cerrado (**closed**).

Es importante detallar que los valores (a menos que se indique lo contrario) solo pueden ser reales. Además, los modificadores pueden ir desde **nano** hasta **giga**, siendo **base** la opción sin modificador (la opción por defecto). Por último, se debe tener en cuenta que para los campos de texto se acepta tanto la opción toda en minúscula como la toda en mayúscula o aquella que solo empieza por mayúscula.

Por otro lado, tenemos la estructura **Parallel**, cuyo contenido es un conjunto de al menos dos subestructuras **Branch**. Dentro de cada **Branch** se declara un componente, de igual manera que los descritos anteriormente.

3. Implementación

El desarrollo del compilador se realizó en lenguaje C, dividido en Frontend y Backend, a partir del repositorio base provisto por la cátedra. El sistema utiliza Flex y Bison para procesar el

código fuente y construir un Árbol de Sintaxis Abstracta (AST), estructura que posteriormente es recorrida para verificar la consistencia semántica y generar, como resultado final, el código LaTeX/CircuiTikZ necesario para la graficación del circuito. Para aclarar, de ahora en adelante se le llamara una restricción “sintáctica” a todas las restricciones que hemos definido a través de nuestra gramática, y restricción “semántica” a las restricciones que hemos definido a través del parseo y construcción de nuestra tabla de símbolos, verificados directamente dentro del sector de Backend en el documento EntryPoint.c.

3.1 CircuiTikZ

Antes de pasar al proceso de implementación del lenguaje y los problemas que nos encontramos en el proceso, resulta importante hacer aclaraciones sobre el lenguaje de programación usado como nuestra salida para después graficar los circuitos: el paquete de LaTeX llamado **CircuiTikZ**.

Esta librería utiliza TikZ, otro paquete de LaTeX que se utiliza para poder generar gráficos con vectores. En grandes rasgos, se trata de definir las líneas del circuito a base de coordenadas cartesianas. Por ejemplo, el siguiente código:

```
(0,0) to (0,4)
(0,4) to (4,4)
(4,4) to (4,0)
(4,0) to (0,0)
```

Código 1. Código CircuiTikZ para dibujar un circuito sin ningún componente.

genera un cuadrado 4x4 sin componentes. En el caso de querer dibujar un circuito con un elemento paralelo, tendría que ver qué líneas tendría que dibujar; no existe una manera directa de agregar un paralelo y sus ramas.

Para agregar componentes, se puede especificar posterior a la keyword `to` que componente quiero agregar, también pudiendo agregarle un label (que puede ser tanto un texto como un valor). Por ejemplo, el siguiente código:

```
(0,0) to[battery] (0,4)
(0,4) to (4,4)
(4,4) to[eC, l=$0.2mF$] (4,0)
(4,0) to (0,0)
```

Código 2. Código CircuiTikZ para dibujar un circuito con batería y capacitor con valor.

genera un circuito con una batería y un capacitor con la polaridad y capacitancia indicada de la siguiente manera:

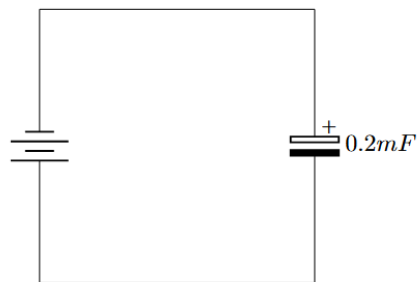


Figura 1. Circuito con batería y capacitor, generado por el código 2.

Es importante mencionar una limitación que tiene CircuiTikZ en cuanto al dibujo de una componente, y es que se puede incluir solamente un componente por intervalo. Es decir, si yo quiero agregar dos componentes en el intervalo $(0,0)$ to $(0,4)$, debo hacerlo de la siguiente manera:

```
(0,0) to[battery] (0,2)
(0,2) to[capacitor] (0,4)
```

Código 3. Código CircuiTikZ para agregar un lado de circuito con una batería y un capacitor.

Esto implica que agregar múltiples componentes a un lado de una malla es complejo al tener que dividir ese mismo lado en la cantidad de componentes que se quisieran agregar.

Si bien las limitaciones del paralelo y los componentes mencionados en este sector fueron razones por la cual se restringieron aspectos del lenguaje, las ventajas que ofrece CircuiTikZ en su alcance significan que hay mucho que se podría agregar al lenguaje en el futuro.

3.2 Frontend

Al empezar a diseñar el lenguaje, se encontraron varias limitaciones en lo que iba a ser nuestro dominio de circuitos debido al funcionamiento del paquete CircuiTikZ. La limitación de no poder agregar directamente dos componentes en un mismo intervalo de manera directa fue lo que nos llevó a la decisión de la primera restricción sintáctica: **no se puede utilizar el lenguaje para dibujar circuitos con más de cuatro elementos en la malla principal**. Es decir, el siguiente circuito **no** estaría dentro del dominio de nuestro lenguaje:

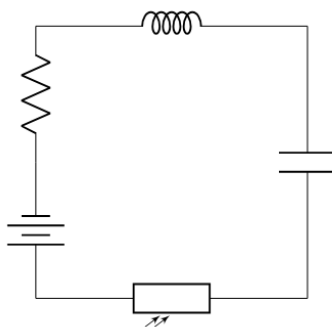


Figura 2. Circuito fuera de dominio por exceso de componentes en malla principal.

Es importante destacar el uso de la palabra “elementos” dentro de la restricción en vez de “componentes”. Esto significa que tampoco podría agregar cuatro componentes y también un lado con ramas paralelas a ese lado, ya que el agregar un paralelo también contaría como un elemento. La manera en la que se verificó esta restricción fue simplemente agregarle las siguientes producciones a nuestra gramática, asegurando el máximo de cuatro elementos por malla principal:

```
elementList → element
elementList → element COMMA element
elementList → element COMMA element COMMA element
elementList → element COMMA element COMMA element COMMA element
```

Código 4. Producciones para generar la malla principal del circuito.

donde el símbolo no terminal `elementList` se deriva de la producción del símbolo no terminal `circuit`.

En cuanto al paralelo, la necesidad de tener que dibujar las ramas intervalo por intervalo en vez de poder usar alguna funcionalidad directa de paralelo nos llevó a las dos siguientes restricciones sintácticas: **se puede tener un máximo de un solo componente por rama en un paralelo** y **no se pueden anidar paralelos**. Los dos siguientes circuitos, por ejemplo, serían inválidos:

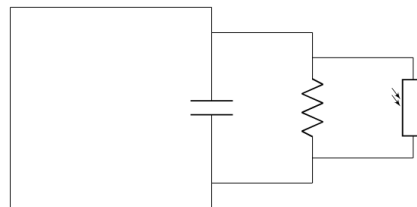


Figura 3. Circuito fuera del dominio por un paralelo anidado.

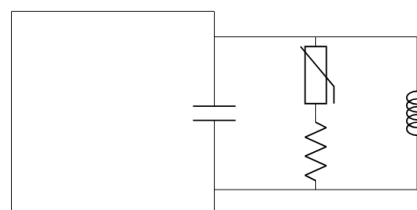


Figura 4. Circuito fuera de dominio por tener dos componentes dentro de una rama.

La manera en la que se verificó ambas restricción desde la gramática fue con la siguiente producción:

```
componentList → component
```

Código 5. Producción para generar el contenido de cada rama dentro de un paralelo.

donde el símbolo no terminal **componentList** se deriva de la producción del símbolo no terminal **branch**. Si bien no termina siendo una lista ya que es un máximo de un componente, se define de tal manera como para poder extenderlo en un futuro si se quieren agregar más componentes a una rama.

La razón por la cual es esta producción que restringe ambas cosas es porque, por un lado, solo admite que el contenido del mismo sea un componente y no un elemento, excluyendo el paralelo. En cuanto a la cantidad de componentes, es simple ver que solamente se admite un componente por rama.

En rasgos generales, el resto de la gramática consiste en asignarle a cada keyword (excluyendo **Parallel**) con un identificador, y en el caso de los componentes, una serie de parámetros los cuales tenían cada uno su propio grupo de producciones. Esto se eligió así para poder diferenciar bien entre las funcionalidades que podrían llegar a tener cada uno de los componentes. Aunque con el alcance actual algunos de los parámetros entre componentes parecen quedar redundantes (por ejemplo, el símbolo no terminal **directSourceParams** tiene las mismas producciones que **capacitorParams**), se hizo la diferenciación igual así la gramática no cambiaba drásticamente si se tenía que hacer alguna modificación a los parámetros de algún componente.

Por último, es relevante aclarar una decisión en específico que se tomó, y es interpretar el token **COMPLEX_VALUE_TOKEN** como un valor en conjunto en vez de que sea una parte real y otra imaginaria. Esto se hizo teniendo en cuenta el alcance que iba a tener el lenguaje actualmente; ya que no incluimos ningún aspecto del cálculo de los valores de voltaje y corriente dentro de los circuitos, nos pareció mejor mantener la coherencia de la gramática a complejizar este aspecto teniendo en cuenta que no se iba a aprovechar la ventaja de separar el token en dos valores.

3.3 Backend

Lo primero relevante a mencionar del Backend sería el diseño de nuestra tabla de símbolos. Si bien la planteamos inicialmente como una tabla más compleja, llegamos a la conclusión de que la estructura necesaria para poder cumplir con las restricciones actuales era la siguiente:

```
typedef struct Declaration {
    const char * scopeId;
    const char * id;
    TypeEnum type;
} Declaration;
```

Código 6. La estructura 'Declaration', la cual contiene cada columna de nuestra tabla de símbolos

El **scopeId** era necesario para poder ver dentro de qué circuito se encontraba cada componente/rama (siendo los atributos que podían poseer identificador), y en el caso de que la declaración consiste de agregar un circuito, esta columna se dejaba en **null**. El **id** se guardaba para verificar una de las restricciones que será especificada en breve, y el **type** marcaba qué atributo se estaba guardando en la tabla (**Circuit**, **Branch**, **Resistor**, etc). Si bien este último podría haber simplemente guardado un tipo **Component** para todos los componentes, elegimos

hacer la diferenciación para cada uno así en futuras extensiones este cambio no era necesario hacerlo, teniendo en cuenta que hacerlo así desde un inicio no significaba una dificultad muy elevada y tampoco una pérdida de eficiencia. Estas declaraciones después se guardaban en la estructura de `SymbolTableCDT`, que tenía la siguiente definición:

```
typedef struct SymbolTableCDT {
    List declarations;
    bool redefinition;
    bool differentSourceType;
} SymbolTableCDT;
```

Código 7. La estructura ‘SymbolTableCDT’, la cual contiene la lista de declaraciones (tuplas) y booleanos para verificar restricciones

También vimos la necesidad de hacer algunas restricciones semánticas al lenguaje. Sin embargo, en este caso no fueron causadas por las limitaciones de `CircuitikZ` como en el Frontend, sino por decisiones del diseño del lenguaje para mejorar su legibilidad y también respetar las cualidades necesarias para formar un circuito de verdad.

La primera restricción semántica fue la siguiente: **múltiples circuitos no pueden tener el mismo identificador, y componentes dentro de un mismo circuito tampoco pueden tener el mismo identificador**. Es decir, el siguiente programa sería inválido:

```
Circuit C1 {
    Resistor R1 (regular),
    Resistor R1 (potentiometer, 100, micro) ← INVALIDO, redefinición de R1
},

Circuit C1 { ← INVALIDO, redefinición de C1
    Resistor R1 (thermistor, 10) ← VÁLIDO, es un identificador repetido pero
    en otro circuito
}
```

Código 8. Código inválido por redefinición de identificadores.

También es importante aclarar el comentario que se hace dentro del segundo circuito definido, enfatizando que los identificadores de componentes **si** se pueden repetir si simplemente no se repiten dentro del mismo circuito. La manera en la que se verificó esta restricción fue a través de él flag booleano `redefinition` dentro de la estructura de la tabla de símbolos. Al ir agregando declaraciones a nuestra tabla de símbolos, se fijaba si había algún otro componente dentro del mismo circuito (o programa en el caso de identificadores de circuitos) usando `scopeId` que poseía el mismo identificador. En el caso de que lo encontrara, este booleano se pondría en `true`, y al entrar en el análisis semántico de `EntryPoint.c` se verificará el valor de este booleano.

La segunda restricción a tener en cuenta es la siguiente: **no se puede tener tipos distintos de fuentes dentro de un mismo circuito**. Es decir, si bien yo puedo tener múltiples fuentes dentro de un mismo circuito, no puedo mezclar entre fuentes de corriente alterna y corriente directa. El objetivo de esta restricción fue más que nada respetar el contexto real de la construcción de circuitos, ya que no se podría mezclar fuentes de esta manera al hacer un

circuito de verdad. El código inválido que limita esta restricción sería el siguiente:

```
Circuit C1 {  
    DCSource DC1 (100, micro),  
    ACSource AC1 (10-10i, micro) ← INVALIDO  
}
```

Código 9. Código inválido por mezcla de fuentes de corriente directa y alterna dentro de un mismo circuito.

La manera en la que se verificó esta restricción fue usando el flag booleano **differentSourceType** dentro de la estructura de la tabla de símbolos de una manera similar al uso del flag anterior; se verifica cuál fue el tipo de fuente que se agregó primero en la tabla de símbolos, y si se agrega un tipo de fuente distinto a este, el booleano se pondría en **true**.

3.4 Dificultades Encontradas

Fuera de las dificultades técnicas que son comunes con este tipo de proyectos (mayormente la cohesión y adaptación al código base que fue entregado para realizar el proyecto), la mayor dificultad que nos encontramos a lo largo de todo el desarrollo fue el diseño del lenguaje, y su evolución con el tiempo al analizar las herramientas a disposición más en detalle.

El lenguaje se había planteado desde un comienzo con un dominio mucho superior al final, ya que si bien se había planteado el uso de CircuitikZ como herramienta para después dibujar los circuitos, no se llevó un análisis extensivo de su uso y limitaciones que después llevaron a lo que es hoy el dominio. Este análisis se llevó a cabo más que nada en el comienzo del desarrollo del Backend, por lo cual también hubo un cambio significativo entre la segunda y tercera etapa.

Parte de esta dificultad también fue no restringir el dominio demasiado para poder mantener el objetivo final de este lenguaje: poder crear circuitos de una manera mucho más simple y directa. Limitar demasiado el dominio no perdería de foco la simplicidad, pero si la utilidad que un usuario le podría encontrar al lenguaje, así que se analizaron varias opciones y se llegó a la conclusión de que las que se plantearon a lo largo del informe fueron las justas y necesarias para poder implementar el lenguaje de la mejor manera posible y manteniéndolo lo suficientemente útil para los usuarios.

4. Futuras Extensiones

Si bien el lenguaje actual es funcional dentro de su dominio acotado, el desarrollo evidenció oportunidades claras para aumentar su expresividad, flexibilizando las restricciones topológicas impuestas.

El principal objetivo para una futura versión sería eliminar la limitación de un único componente por segmento. Esto implicaría permitir conexiones en serie (múltiples componentes consecutivos) tanto en los lados de la malla principal como dentro de las ramas de un paralelo.

Asimismo, se buscaría habilitar el soporte para estructuras anidadas, permitiendo declarar bloques **Parallel** dentro de un **Branch**. Implementar esto requeriría una reingeniería significativa del Backend para pasar de un cálculo de coordenadas estático a un sistema dinámico capaz de ajustar el tamaño de la malla y la posición de los nodos automáticamente según la profundidad y complejidad del circuito descrito.

Por otro lado, impulsando aún más las posibilidades del dominio, sería interesante incluir validaciones de los valores e incluso cálculo de incógnitas o valores faltantes. De esta manera, el compilador no solo permitiría modelar y graficar problemas de física, sino que también resolverlos. Esta funcionalidad se podría agregar directamente a los componentes de **Voltmeter** y **Ammeter**, ya que actualmente no tienen ninguna manera de atribuirles valor, y el valor calculado podría llegar a ser este mismo.

Por último, lo que no llegamos a corregir a partir de las correcciones del Stage II fue la manera en la que se construían todas las listas que utilizamos dentro de nuestra gramática. Esto fue algo que no pudimos corregir por cuestiones de tiempo ya que implicaba un cambio grande dentro de nuestras acciones semánticas y las estructuras de cada lista, pero es una extensión importante para hacer pensando en el resto de las extensiones que se podrían hacer al lenguaje por una cuestión de eficiencia.

5. Conclusiones

La realización de este proyecto nos permitió consolidar los conocimientos teóricos sobre compiladores, llevando conceptos abstractos a una implementación funcional. El lenguaje desarrollado cumplió con su objetivo principal: abstraer la complejidad de LaTeX para simplificar el diseño de circuitos.

En cuanto a lo personal, el desarrollo de este proyecto nos pareció una gran experiencia. Fue enriquecedor ver todo el ciclo de compilación en acción y comprender cómo un código escrito en nuestro lenguaje era procesado e interpretado correctamente para generar resultados visuales complejos. Jugar con las limitaciones de nuestro lenguaje y ver plasmados los resultados en los distintos circuitos fue muy gratificante. Además, sentimos que fue una gran manera de utilizar el contenido de la materia para desarrollar algo real que además podría ser usado por cualquier persona para dibujar sus propios circuitos.

En síntesis, aunque nos hubiera gustado que nuestro lenguaje tuviera la potencia planteada desde un inicio, aprendimos mucho haciendo este trabajo, y estamos muy contentos con el resultado final.

6. Referencias

- [1] Massimo A. Redaelli, Stefan Lindner, Stefan Erhardt, and Romano Giannetti. *CircuitikZ: Draw electrical networks with TikZ*. CTAN, The Comprehensive TeX Archive Network. Disponible en: <https://ctan.org/pkg/circuitikz>.
- [2] Cátedra de Autómatas, Teoría de Lenguajes y Compiladores. *Flex-Bison-Compiler*. Disponible en: <https://github.com/agustin-golmar/Flex-Bison-Compiler>.

7. Bibliografía

- Material provisto por la cátedra de Autómatas, Teoría de Lenguajes y Compiladores.